Problem Statement:

ebay is an online shopping site that is known for its auctions and consumers to consumer sales. ebay has an online marketplace where you can buy, sell items through auctions where the price is decided by bidding or through a marketplace at a fixed price.

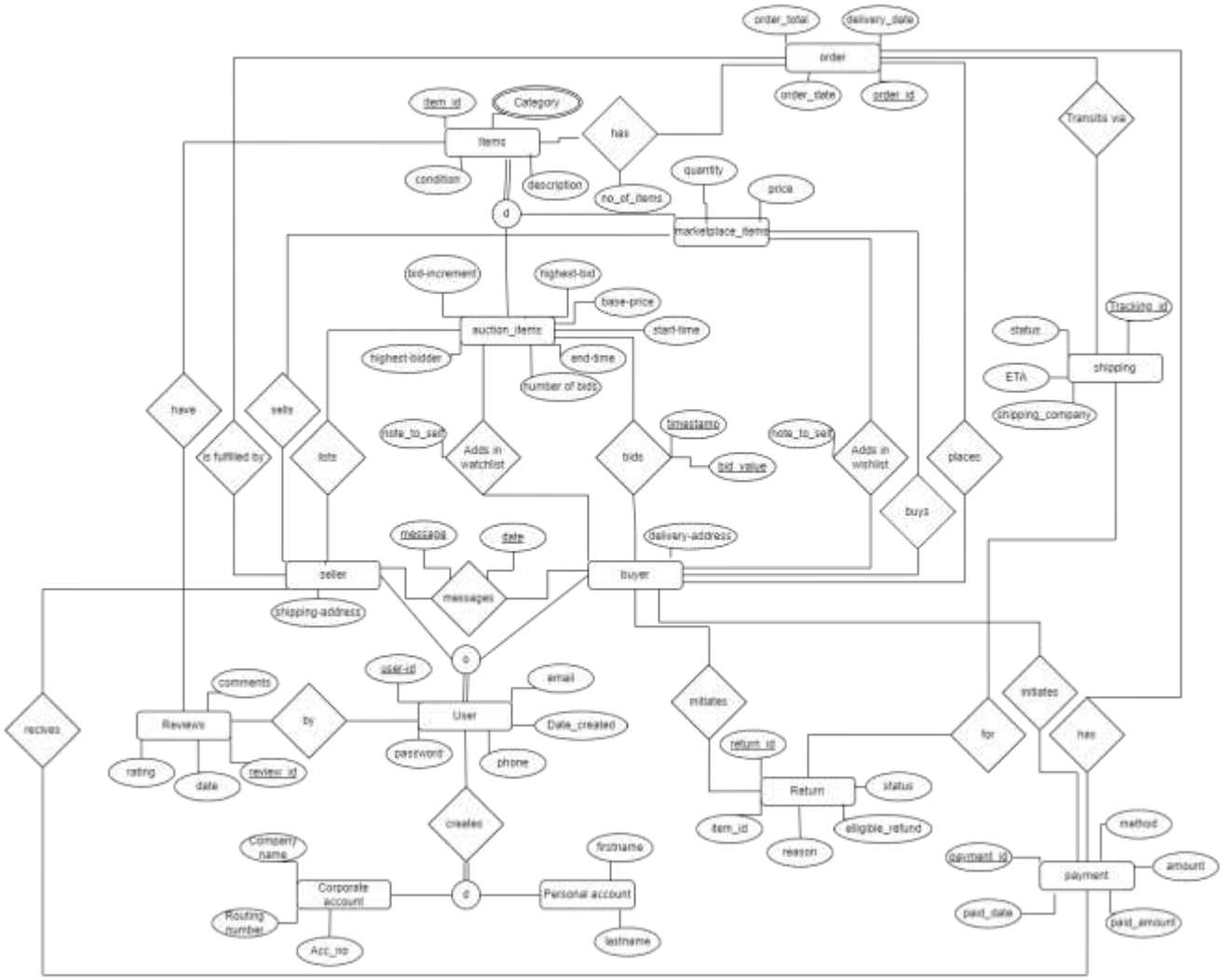
Our goal is to design a database to support the services provided by eBay. We will use oracle(sql) database for the purpose of this project.

Data Requirements:

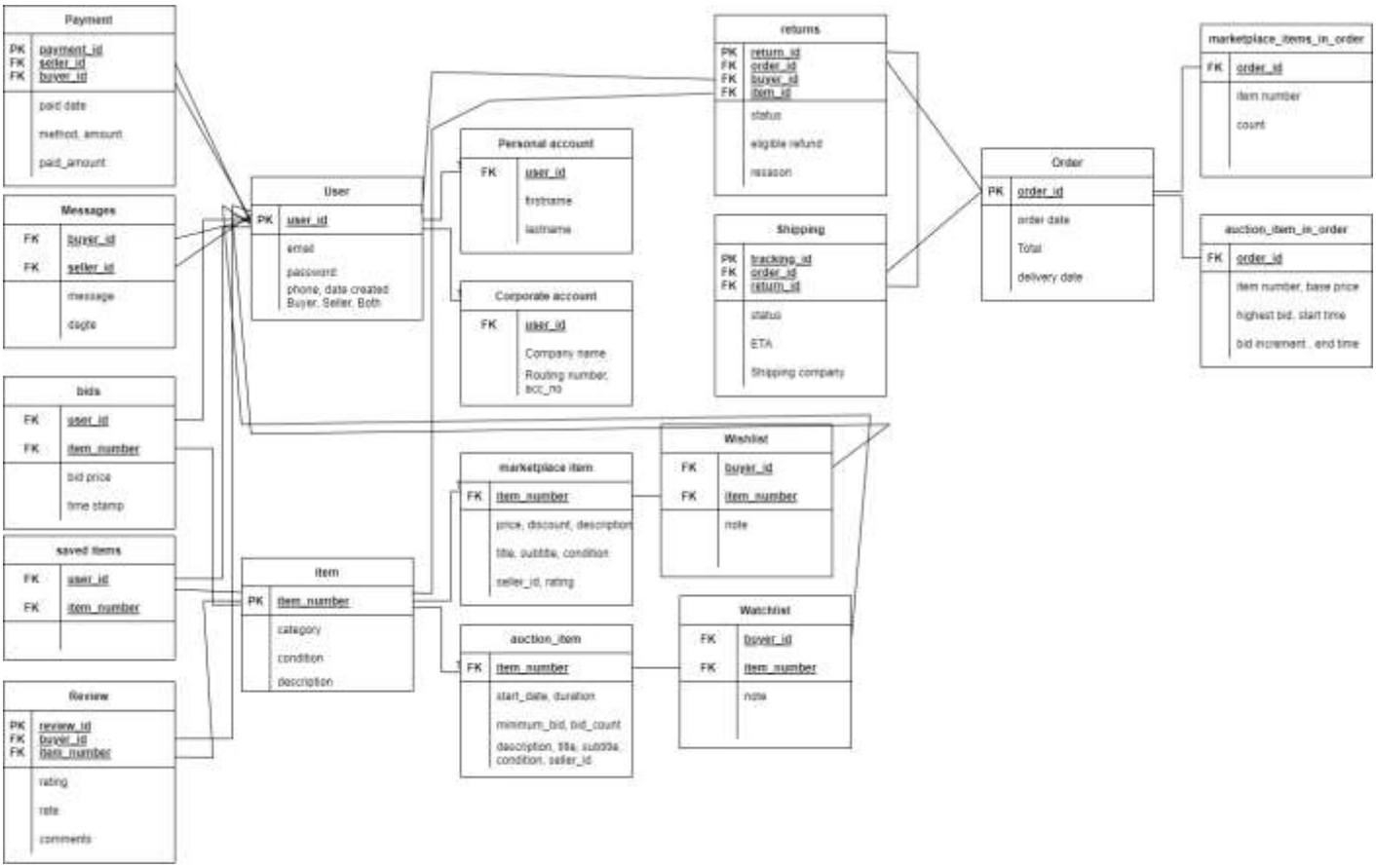
- As an online marketplace, eBay will have users and every user can create one user account with eBay. Each entry in the **User** table will have unique user_id, email, date_created, phone, and a password. user_id will be the primary key to recognize each user. This will also help maintain the user personal information anonymous with others.
- User can create a **personal account** or a **corporate account**. Personal account will have firstname, lastname whereas corporate account will have company name, routing number, acc_no. A user in users table can be a buyer or seller or both.
- Buyer and seller can message each other. Every **message** has subject, message, and date.
- eBay has **items** that it needs to sell, every item has unique item number, description, subtitle, condition, title and category. The same item can have multiple categories . Item number will act as the primary key. Item_id is the primary key for items table.
- Every item is divided in to either as a **marketplace_item** or a **auction_item**. A Buyer can buy an item that is a marketplace_item for the price set by the sell as long as it is available, whereas for a auction_item buyer has to successfully bid for that item and win the bid when the auction is active.
- auction_items have base_price, no of bids, start time and end time, highest_bidder, bid_increment. market_place_item has price, quantity.
- buyer can bid on multiple auction_item and on each auction_item multiple buyers can place bid. Each **bid** has bid amount and bid time.
- When the end time for a bid is reached, the buyer with the highest bid will receive the item. auction_item will be added to the **order** and **payment** should be collected form the winning bidder. A bid will be void after the end time if no one bids on it and will be marked unsold.

- A buyer can place multiple orders, each which will have a unique order_id, order total, order date and delivery date.
- Every order contains either at least one marketplace_item and count of marketplace_item or exactly one auction_item. Each marketplace_item can be part of multiple orders while each auction_item will be part only one order.
- Payment will be done for each order and vice versa. Payment has unique payment ID, payment method, amount, credit or debit card number and date of payment.
- Seller receives many payments, but every payment is done to one seller only.
- One user can watch multiple items to help keep track of the price or bid price of an item. Each item can be watched by multiple users at the same time.
- Users can give **reviews** to the marketplace_item. Review will have unique reivew_id, rating, date and comments.
- Each marketplace_item can have multiple Reviews given by users and every review is associated with one marketplace_item.
- Orders that need to be returned are tracked as part of the **returns** table. Returns will have return_id, status, eligible refund, reason, item_id.
- **Shipping** table keeps track of all the orders and returns shipping. Tracking_id, status, ETA, shipping_company

ER Diagram:


order_total
delivery_date
order
order_date
order_id
Transits via
item_id
Category
Items
has
condition
description
quantity
no_of_items
price
d
marketplace_items
bid-increment
highest-bid
base-price
start-time
auction_items
highest-bidder
end-time
number of bids
Tracking_id
status
ETA
shipping
shipping_company
have
sells
is fulfilled by
lists
note_to_self
Adds in watchlist
bids
timestamp
bid_value
note_to_self
Adds in wishlist
places
buys
message
date
delivery-address
seller
buyer
messages
shipping-address
o
user-id
email
User
Date_created
password
phone
recives
comments
Reviews
by
rating
date
review_id
creates
Consent/ name
Corporate account
Routing number
Acc_no
d
firstname
Personal account
lastname
initiates
return_id
Return
status
item_id
reason
eligible_refund
for
initiates
has
method
payment_id
payment
amount
paid_date
paid_amount

Relational Schema:



Normalization:

The above relations are already in 1NF, 2NF and 3NF form. All the normalization rules are followed. Therefore, there is no further need of normalization.

SQL Commands:

```
CREATE TABLE USER (

User_id varchar(15) primary key,

Phone_number varchar(50),

Date_created date NOT NULL,

Address varchar(50) NOT NULL

);


Insert INTO USER values('Tejo_vardhan', '+1(469)-159-7410', NOW(), '7854 P george bush Rd, Dallas, Texas, 7894');

Insert INTO USER values('Manasi', '+1(469)-645-5647', NOW(), '9020 Garland Rd, Dallas, TX 75218');

Insert INTO USER values('Yeshwanth_kuchimanchi', '+1(469)-265-6789', NOW(), '7621 Donald Trump Rd, San antonio, Texas, 7254', 'yeshwanthemail@gmail.com');


CREATE TABLE PERSONAL_ACCOUNT (

Email varchar(30) primary key,

User_id varchar(15),

Password varchar(50) check(length(password)>8),

Fname varchar(20) NOT NULL,

Lname varchar(20) NOT NULL,

foreign key (User_id) references USER (User_id) on delete CASCADE

);


Insert INTO PERSONAL_ACCOUNT values('tejovardhan100@hotmail.com', 'Tejo_vardhan', ‘awe#_456@SRY’, 'Tejo', 'M');

Insert INTO PERSONAL_ACCOUNT values('manasi@yahoo.com', 'Manasi', 'khf@_#isbg&j134', 'Manasi', 'B');
```

```
CREATE TABLE CORPORATE_ACCOUNT (
Email varchar(30) primary key,
User_id varchar(15),
Password varchar(50) check(length(password)>8),
Company_name varchar(20) NOT NULL,
Acount_number varchar(11) NOT NULL,
Routing_number varchar(8) NOT NULL,
foreign key (user_id) references users (user_id) on delete CASCADE
);

Insert INTO CORPORATE_ACCOUNT values('yeshwanthemail@gmail.com', 'Yeshwanth_kuchimanchi',
'sfd#fsg1332@-ssSdfg', 'Innovative tech', '4567891230', '6513247894');

CREATE TABLE MARKETPLACE_ITEMS (
Item_number char(10) primary key,
Description varchar(50),
Quantity varchar(50)
Price numeric(10,2) NOT NULL,
Discount numeric(2),
Condition varchar(15) check(Condition in('Used', 'New', 'Refurbished', 'Openbox')),
Rating decimal(2,1) check(Rating < 5.0)
Seller_id varchar(15),
foreign key(Seller_id) references USER (User_id) on delete CASCADE
);

INSERT INTO MARKETPLACE_ITEMS values('56128Z', 'quadcopter', 2, 15.99, 20, 'New', 4.8, 'yxk');
CREATE TABLE AUCTION_ITEMS (
Item_number char(10) primary key,
Title varchar(30) NOT NULL,
Description varchar(50),
Condition varchar(15) check(Condition in('Used', 'New', 'Refurbished', 'Openbox')),
```

```
Number_of_bids int DEFAULT 0

Minimum_bid numeric(10,2) NOT NULL,

Highest_Bid varchar(20),

Start_time timestamp NOT NULL,

end-time timestamp NOT NULL,

Seller_id varchar(15),

Highest_Bidder varchar(15),

foreign key(Seller_id) references USER (User_id) on delete CASCADE,

foreign key(Highest_Bidder) references USER (User_id) on delete CASCADE

);

INSERT INTO AUCTION_ITEMS values('4563223', ‘Maserati levante', 'Luxury vehicle', 'Used', 16, 30000, 75000,

TO_DATE('04-05-2022 12:30:15', 'YYYY-MM-DD HH:MI:SS'), TO_DATE('04-05-2022 12:30:15', 'YYYY-MM-DD HH:MI:SS'), 'Yeshwanth_kuchimanchi', 'Tejo_vardhan');


CREATE TABLE MARKETPLACE_ITEMS_CATEGORY (

Item_number char(10) primary key,

Category varchar(20) primary key,

foreign key(Item_number) references MARKETPLACE_ITEMS (Item_number) on delete CASCADE

);

INSERT INTO MARKETPLACE_ITEMS_CATEGORY values('54698712', 'Cosmetics');


CREATE TABLE AUCTION_ITEMS_CATEGORY (

Item_number char(10) primary key,

Category varchar(20) primary key,

foreign key(Item_number) references BID _ITEM (Item_number) on delete CASCADE

);


INSERT INTO AUCTION_ITEMS_CATEGORY values(‘1123985750’, 'Vehicle');


CREATE TABLE ORDER (

Order_id char(10) primary key,
```

```
Order_total numeric(10,2) NOT NULL,
Order_date date NOT NULL,
Delivery_date date,
Buyer_id varchar(15),
foreign key(Buyer_id) references USER (User_id) on delete CASCADE);

INSERT INTO ORDER values('56128Z', 38.98, TO_DATE('05-05-2022', 'dd-mm-yyyy'),
TO_DATE('07-05-2022', 'dd-mm-yyyy'), 'Manasi');

CREATE TABLE MARKETPLACE_ITEMS_IN_ORDER (
Order_id
Item_number
char(10)
char(10)
primary key,
primary key,
Count int DEFAULT 1,
foreign key(Order_id) references ORDER (Order_id) on delete CASCADE
foreign key(Item_number) references MARKETPLACE_ITEMS (Item_number)
);

CREATE TABLE AUCTION_ITEMS_IN_ORDER (
Order_id char(10) primary key,
Item_number char(10) primary key,
foreign key(Order_id) references ORDER (Order_id) on delete CASCADE
foreign key(Item_number) references AUCTION_ITEMS (Item_number)
);

CREATE TABLE BIDS (
Item_no char(10) primary key,
User_id varchar(15) primary key,
```

```
Bid_value numeric(10,2) NOT NULL
Bid_time timestamp default sysdate,
foreign key(Item_no) references AUCTION_ITEMS (Item_number),
foreign key(User_id) references USER (User_id) on delete CASCADE,
);

CREATE TABLE PAYMENT (
Payment_id char(10) primary key,
paid_amount numeric(10,2) NOT NULL,
Payment_method varchar(20) NOT NULL,
Date date,
Order_id char(10),
Seller_id varchar(15),
foreign key(Order_id) references Order (Order_id) on delete CASCADE,
foreign key(Seller_id) references USER (User_id) );

CREATE TABLE Review (
review_id char(8) primary key,
Buyer_id varchar(15),
Item_no char(10),
Rating int NOT NULL
Headline varchar(20)
Review varchar(50),
Date date,
check(Rating >= 0 AND Rating <= 5),
foreign key(Buyer_id) references USER (User_id),
foreign key(Item_no) references MARKETPLACE_ITEMS (Item_number) on delete CASCADE
);

CREATE TABLE MESSAGE (
Message varchar(100)
```

```
Sender_id varchar(15) primary key,
Receiver_id varchar(15) primary key,
Subject varchar(50),
Date timestamp primary key,
foreign key(Sender_id) references USER (User_id) on delete CASCADE,
foreign key(Receiver_id) references USER (User_id) on delete CASCADE
);

CREATE TABLE SAVED_ITEMS (
User_id varchar(15) primary key,
Item_no char(10) primary key,
foreign key(User_id) references USER (User_id) on delete CASCADE,
foreign key(Item_no) references MARKETPLACE_ITEMS (Item_number) on delete CASCADE);
```

Procedures:

1. The below is a procedure to get winner of the auction for an acution item after the auction time is completed, i.e after end_time of auction. Once the auction is finished, the auction item will be added to order and will be removed from the auction_items table.

```
SET SERVEROUTPUT ON;
CREATE or REPLACE PROCEDURE winning_bid(item_number IN AUCTION_ITEMS.Item_number%type) AS
thisEnddate AUCTION_ITEMS.Start_date%TYPE;
thisMaxBid BIDS.Bid_amount%TYPE;
thisBuyer BIDS.User_id%TYPE;
thisOrderId ORDER.Order_id%TYPE;
BEGIN
        select Start_date + Duration INTO thisEnddate from AUCTION_ITEMS where Item_no =
item_number;
        select max(Bid_amount) INTO thisMaxBid from BIDS where Item_no = item_number;
        select User_id INTO thisBuyer from BIDS where Bid_amount = thisMaxBid and Item_no
        = item_number;
        IF sysdate > thisEnddate THEN
                LOOP
                        select round(DBMS_Random.Value(0000000001,9999999999)) INTO
                        thisOrderID from dual;
                        EXIT WHEN thisOrderID NOT IN ORDER.Order_id;
                END LOOP
                        INSERT INTO Order values(thisOrderId, thisMaxBid, sysdate, sysdate + 3,
                        thisBuyer);
                        INSERT INTO BID_ITEMS_IN_ORDER values(thisOrderId, item_number);
                        DELETE FROM BID_ITEM where Item_no = item_number;
                        DELETE FROM BIDS where Item_no = item_number;
        END IF
END
```

2. The below is a procedure that will calculate an item rating from the Review given by the users and updates the item table with the latest rating.

```
SET SERVEROUTPUT ON;
CREATE or REPLACE PROCEDURE calculate_rating(item_number IN Review.Item_no%TYPE)
AS
thisRating MARKETPLACE_ITEMS.Rating%TYPE;
sum numeric(5,1);
count numeric(5,0);
rating MARKETPLACE_ITEMS.Rating%TYPE
CURSOR calculate_rating IS
        select Rating from Review where Item_no = item_number;
BEGIN
        OPEN calculate_rating;
                sum := 0.0;
        LOOP
                FETCH calculate_rating INTO thisRating;
                EXIT WHEN calculate_rating%NOTFOUND;
                sum := sum + thisRating;
        END LOOP;
        count := calculate_rating%ROWCOUNT;
        select Rating INTO rating from MARKETPLACE_ITEMS where Item_no = item_number;
        UPDATE MARKETPLACE_ITEMS set Rating = (sum+rating)/(row_count+1) where Item_no =
        item_number;
        dbms_output.put_line('Rating updated for item: ' || item_number);
        CLOSE calculate_rating;
END;
```

3. The below is a procedure that will calculate the next minimum bid price to be made based on the current price which is the last highest bid and updates the minimum bid on the auction_item table.

```
SET SERVEROUTPUT ON;

CREATE or REPLACE PROCEDURE update_minbid(item_number IN BIDS.Item_no%TYPE,

bid_price IN

BIDS.Bid_amount%TYPE) AS

        minBidPrice AUCTION_ITEMS.Minimum_bid%TYPE;

BEGIN

        IF bid_price>= 0.01 AND bid_price<= 0.99 THEN

                minBidPrice := bid_price+ 0.05;

        ELSIF bid_price>= 1.00 AND bid_price<= 4.99 THEN

                minBidPrice := bid_price+ 0.25;

        ELSIF bid_price>= 5.00 AND bid_price<= 24.99 THEN

                minBidPrice := bid_price+ 0.50;

        ELSIF bid_price>= 25.00 AND bid_price<= 99.99 THEN

                minBidPrice := bid_price+ 1.00;

        ELSIF bid_price>= 100.00 AND bid_price<= 249.99THEN

                minBidPrice := bid_price+ 2.50;

        ELSIF bid_price>= 250.00 AND bid_price<= 499.99 THEN

                minBidPrice := bid_price+ 5.00;

        ELSIF bid_price>= 500.00 AND bid_price<= 999.99 THEN

                minBidPrice := bid_price+ 10.00;

        ELSE

        END IF;

        minBidPrice := bid_price+ 25.00;

        END;

        UPDATE AUCTION_ITEMS set Minimum_bid = minBidPrice where Item_no = item_number;

        UPDATE AUCTION_ITEMS set Bid_count = Bid_count + 1 where Item_no = item_number;

END;
```

Triggers:

1. The below trigger will calculate the average rating of the items based on Review given by the user and will go and will be triggered whenever buyer gives review to an item. This trigger will call the second procedure.

```
CREATE OR REPLACE TRIGGER rating_update
        AFTER INSERT ON Review FOR EACH ROW
DECLARE
        PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
        calculate_rating(:NEW.Item_no);
        COMMIT;
END;
```

2. The below trigger that will check if a new bid is made that is greater than current minimum bid. If yes, then it will calculate the new minimum bid and update it in the bid_items table. This trigger will be made whenever a new bid is made by a bidder for an item. Trigger action will call third procedure.

```
CREATE OR REPLACE TRIGGER update_bid_details
AFTER INSERT ON BIDS
FOR EACH ROW
DECLARE
        PRAGMA AUTONOMOUS_TRANSACTION;
        thisMinBid AUCTION_ITEMS.Minimum_bid%TYPE
BEGIN
        SELECT Minimum_bid into thisMinBid from AUCTION_ITEMS where Item_no = :NEW.Item_no;
        IF :NEW.Bid_amount < thisMinBid THEN
                Raise_Application_Error(-20000, 'You have to bid atleast ' || thisMinBid);
        Else
                update_minbid(:NEW.Item_no, :NEW.Bid_amount);
        END IF
        COMMIT;
END;
```